

JavaScript Questions part-2

1.What are Promises in JavaScript?

Promises are a way to handle asynchronous operations in JavaScript — such as data fetching, timers, or reading files — without falling into deeply nested callbacks (a.k.a. "callback hell").

◆ A Promise Represents:

- **Pending:** Initial state, operation not complete yet.
- **Fulfilled:** Operation completed successfully.
- **Rejected:** Operation failed with an error.

A

Promise is an object

- `Promise` is a **built-in class (constructor function)** in JavaScript.
- You **instantiate** it with `new Promise()` .
- It takes a **function (executor)** as its argument.
- That executor function receives two arguments: `resolve` and `reject` .

```
const myPromise = new Promise((resolve, reject) => {  
  let success = true;  
  
  if (success) {  
    resolve("✅ Success!");  
  } else {  
    reject("❌ Error!");  
  }  
});
```

Consuming promise

```

promise
.then((data) => {
  console.log("Success:", data);
})
.catch((error) => {
  console.error("Error:", error);
})
.finally(() => {
  console.log("Operation finished.");
});

```

Real world example

```

function fetchData() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("User data fetched!");
    }, 2000);
  });
}

```

```

fetchData().then((res) => console.log(res)); // Output after 2 sec: User data fetched

```

lets use this example to understand promise better

```

let userLeft = false;
let userWatching = false;

// we have function which takes 2 callback functions as arguments
function watchVideo(callback, errorCallback)
{
  if(userLeft)

```

```

{
  errorCallback({
    name:"user left",
    message:"user signed out"
  })
}else if(userWatching)
{
  errorCallback({
    name:"watching",
    message:"user is watching videos please wait"
  })
}else
{
  callback("success you can subscribe")
}
}

//call the function
watchVideo((message)⇒{
  console.log("message",message);
},(error)⇒{
  console.log(error.name+" "+error.message);
})

//output : - message success you can subscribe

//=== Code Execution Successful ===

```

now we replace this call back code with promise

```

let userLeft = false;
let userWatching = false;

```

```
// now replace this code with promise
function watchVideo()
{
  return new Promise((resolve,reject) =>
  {
    if(userLeft)
    {
      reject({
        name:"user left",
        message:"user signed out"
      })
    }else if(userWatching)
    {
      reject({
        name:"watching",
        message:"user is watching videos please wait"
      })
    }else
    {
      resolve("success you can subscribe")
    }
  })
}

//call the function and resolve promise
watchVideo().then((message) =>{
  console.log("message",message);
}).catch((error) =>{
  console.log(error.name+" "+error.message);
})
```

Callback Hell: What It Looks Like

Let's say you want to simulate a flow like:

1. Check if the user can watch the video
2. Then like the video
3. Then comment on the video
4. Then subscribe to the channel

Each step takes a callback — and soon you're **nesting callbacks inside callbacks**. Here's how your code might look **in callback hell style**:

▼ Modified Code (Callback Hell):

```
let userLeft = false;
let userWatching = false;

function watchVideo(callback, errorCallback) {
  if (userLeft) {
    errorCallback({ name: "user left", message: "user signed out" });
  } else if (userWatching) {
    errorCallback({ name: "watching", message: "user is watching videos, please wait" });
  } else {
    callback("✅ Video watched");
  }
}

function likeVideo(callback) {
  callback("👍 Video liked");
}

function commentOnVideo(callback) {
  callback("💬 Comment added");
}

function subscribe(callback) {
```

```

    callback("🔔 Subscribed");
  }

  // Callback hell starts here 📌
  watchVideo(
    (watchMsg) => {
      console.log(watchMsg);
      likeVideo((likeMsg) => {
        console.log(likeMsg);
        commentOnVideo((commentMsg) => {
          console.log(commentMsg);
          subscribe((subMsg) => {
            console.log(subMsg);
          });
        });
      });
    },
    (err) => {
      console.log(err.name + ": " + err.message);
    }
  );

```

🔍 Output:

✓ Video watched
 👍 Video liked
 💬 Comment added
 🔔 Subscribed

😞 What's Wrong With This?

- **Nested indentation** grows with each new step.

- Harder to **read**, **debug**, and **maintain**.
- If you need to add error handling at each level, it gets even messier.

✓ The Better Way?

Using **Promises** or `async/await`, which flatten the structure.

If you want to run multiple promises at the same time you can use `Promise.all`

```
const videoOne = new Promise((resolve, reject) => {
  resolve('video one uploaded')
})

const videoTwo = new Promise((resolve, reject) => {
  resolve('video two uploaded')
})

const videoThree = new Promise((resolve, reject) => {
  resolve('video three uploaded')
})

//if you want to return only one then which ever promise finishes first
//then use Promise.race
Promise.all([
  videoOne,
  videoTwo,
  videoThree
]).then((messages) => {
  console.log(messages)
})
```

```
// output:[ 'video one uploaded', 'video two uploaded', 'video three uploaded' ]
```

2.How to use async await in JavaScript?

lets say we have this code

`async/await` is **syntactic sugar** built on top of **Promises** that allows you to write asynchronous code in a way that **looks synchronous** and is easier to read and manage.

`async` Keyword

- Used to **declare a function** that returns a Promise.
- Automatically wraps your return value in a Promise.

```
async function greet() {  
  return "Hello!";  
}  
  
greet().then(console.log); // Output: Hello!
```

lets say we have

```
function makeRequest(chatbot)  
{  
  return new Promise((resolve,reject) =>  
  {  
    console.log(`making request to ${chatbot}`)  
    if(chatbot === 'Berry')
```

```

    {
        resolve('Berry says hi')
    }else
    {
        reject("Berry says bye")
    }
})
}

function processRequest(response)
{
    return new Promise((resolve,reject)⇒{
        console.log('Processing response')
        resolve(`Extra Information on chatbot+${response}`)
    })
}

makeRequest("Berry").then(response⇒{
    console.log('Response received')
    return processRequest(response) //return this promise to we can use in .then()
}).then(processResponse⇒{
    console.log(processResponse)
}).catch(err⇒{
    console.log(err)
})

//output:
//making request to Berry
//Response received
//Processing response
//Extra Information on chatbot+Berry says hi

//since its async operation js starts executing next line

async function doWork(){

```

```

//this code should wait till this req is done and then execute it
//to handle errors we use try catch block instead of .then catch
try{
  const response = await makeRequest('Berry')
  console.log('Response Received')
  const processedResponse = await processRequest(response)
  console.log(processedResponse)
}catch(err)
{
  console.log(err);
}
}

```

```

//output after adding this function since calling both functions
//making request to Berry
//making request to Berry
//Response received
//Processing response
//Response Received
//Processing response
//Extra Information on chatbot+Berry says hi
//Extra Information on chatbot+Berry says hi

```

3.What is currying ?

Currying is a process in functional programming in which we can transform a function with multiple arguments into a sequence of nesting functions. It returns a new function that expects the next argument inline.

Let's look at a simple example:

```
function multiply(a, b, c) {
  return a * b * c;
}
```

This function takes three numbers, multiplies the numbers and returns the result.

```
multiply(1,2,3); // 6
```

See, how we called the multiply function with the arguments in full. Let's create a **curried** version of the function and see how we would call the same function (and get the same result) in a series of calls:

```
function multiply(a) {
  return (b) => {
    return (c) => {
      return a * b * c
    }
  }
}
log(multiply(1)(2)(3)) // 6
```

We have turned the `multiply(1,2,3)` function call to `multiply(1)(2)(3)` multiple function calls.

Let's look at another example:

```
function volume(l,w,h) {
  return l * w * h;
}const aCylinder = volume(100,20,90) // 180000l
```

I can design the curried function of *volume* to be this:

```
function volume(l) {
  return (w, h) => {
    return l * w * h
  }
}
```

```
}  
}
```

So it can be called like this:

```
const hCy = volume(70);hCy(203,142);  
hCy(220,122);  
hCy(120,123);
```

or

```
volume(70)(90,30);  
volume(70)(390,320);  
volume(70)(940,340);
```

4.setTimeout and setInterval

We may decide to execute a function not right now, but at a certain time later. That's called "scheduling a call".

There are two methods for it:

- `setTimeout` allows us to run a function once after the interval of time.
- `setInterval` allows us to run a function repeatedly, starting after the interval of time, then repeating continuously at that interval.

These methods are not a part of JavaScript specification. But most environments have the internal scheduler and provide these methods. In particular, they are supported in all browsers and Node.js.

setTimeout

```
function sayHi() {alert('Hello');} setTimeout(sayHi, 1000);
```

With arguments:

```
function sayHi(phrase, who) {alert( phrase + ', ' + who );}  
setTimeout(sayHi, 1000, "Hello", "John");  
// Hello, John
```

If the first argument is a string, then JavaScript creates a function from it.

So, this will also work:

```
setTimeout("alert('Hello')", 1000);
```

But using strings is not recommended, use arrow functions instead of them, like this:

```
setTimeout(() => alert('Hello'), 1000);
```

Canceling with clearTimeout

A call to `setTimeout` returns a “timer identifier” `timerId` that we can use to cancel the execution.

The syntax to cancel:

```
let timerId = setTimeout(...);clearTimeout(timerId);
```

In the code below, we schedule the function and then cancel it (changed our mind). As a result, nothing happens:

```
let timerId = setTimeout(() =>  
  alert("never happens"), 1000);  
alert(timerId); // timer identifier  
clearTimeout(timerId);  
alert(timerId); // same identifier (doesn't become null after canceling)
```

setInterval

The following example will show the message every 2 seconds. After 5 seconds, the output is stopped:

```
// repeat with the interval of 2 seconds
let timerId = setInterval(() => alert('tick'), 2000);
// after 5 seconds stop
setTimeout(() => { clearInterval(timerId); alert('stop'); }, 5000);
```

Nested setTimeout

One is setInterval. The other one is a nested setTimeout, like [this](#):

```
/** instead of:
let timerId = setInterval(() => alert('tick'), 2000);
*/

let timerId = setTimeout(function tick() {
  alert('tick');
  timerId = setTimeout(tick, 2000); // (*)
}, 2000);
```

5.Explain JavaScript Runtime Flow: The Big Picture

JavaScript is **single-threaded**, meaning it can do **one thing at a time** — but it handles async code (like `setTimeout`, Promises, fetch, etc.) using:

Call Stack  Event Loop  Web APIs  Task Queues (Callback & Microtasks)

Step-by-Step Breakdown

1. Call Stack

- It's where **functions are pushed and executed**.
- **Top-down execution**.
- When a function finishes, it's popped off the stack.

```
function sayHi() {  
  console.log("Hi");  
}  
sayHi(); // pushed to stack → runs → popped out
```

2. Web APIs (Provided by Browser)

When async functions like:

- `setTimeout`
- `fetch`
- DOM events
- `setInterval`

are called, they're **not run immediately**.

Instead, they're **handed off to Web APIs**, which handle them **outside the call stack** (in the background).

3. Task Queues

a. Callback Queue / Macrotask Queue

- For tasks like:
 - `setTimeout`
 - `setInterval`

- DOM events
- After the call stack is empty, **Event Loop checks this queue.**

b. Microtask Queue

- For **Promises** (`.then`, `catch`, `finally`), `MutationObserver`, `queueMicrotask`
- Microtasks have **higher priority** than callback/macrotasks.
- **Event loop always drains microtasks first** before moving to callback queue.

4. The Event Loop

The Event Loop continuously checks:

Is the Call Stack empty?

|

- Yes → Are there microtasks?
 - Yes → Run all microtasks
 - No → Run one callback task (e.g. from `setTimeout`)

So the order is:

1. Synchronous code
2. Microtask queue (Promises)
3. Callback queue (`setTimeout`, events)

Example:

```
console.log("Start");

setTimeout(() => {
```

```

    console.log("Timeout");
  }, 0);

  Promise.resolve().then(() => {
    console.log("Promise");
  });

  console.log("End");

```



Output:

Start
 End
 Promise ☒ microtask
 Timeout ☒ callback



Visual Flow Summary

```

Your JS Code →
  Call Stack →
    | if async →
    ↓
  Web APIs →
    wait until ready →
    ↓
  Callback Queue (macrotasks)
    ◀ Event Loop → checks Call Stack → if empty → run microtasks → then 1 call
    back
    ↑

```

Microtask Queue (Promises, etc.)

Real-life Analogy

Imagine a **restaurant**:

-  **Call Stack** = Chef cooking one order at a time
-  **Web APIs** = Waiters handling timers, fetches
-  **Microtask Queue** = Urgent notes from manager (Promises)
-  **Callback Queue** = Regular incoming orders
-  **Event Loop** = Manager checking: "Is the chef free now? What to give next?"

6. Write a function to flatten a nested array

```
function flatten(arr)
{
  let result = [];
  //loop through all values
  for(let i=0;i<arr.length;i++)
  {
    let val = arr[i]
    if(Array.isArray(val))
    {
      //nested values
      //call function again

      let flatVal = flatten(val)
      console.log("flat val", flatVal)
      result=result.concat(flatVal)
    }else
```

```

    {
      result.push(val)
    }
  }

  return result
}

console.log(flatten([1, [2, [3, 4]], 5]));

```

7. Shallow Copy vs Deep Copy

- **Shallow copy** copies only top-level properties
- **Deep copy** copies nested objects as well

```

let obj = { a: 1, b: { c: 2 } };
let shallow = { ...obj }; // changes to b.c affect both
let deep = JSON.parse(JSON.stringify(obj)); // fully independent

```

8. Implement Debouncing

Debouncing means **only run the function after a certain amount of time has passed without the event being triggered again.**

◆ Use case:

- Search box: Wait until the user stops typing before sending API request.
- Window resize: Adjust layout after resizing stops.

```

function debounce(func, delay) {
  let timeoutId;
  return function () {
    clearTimeout(timeoutId);
    const context = this;
    const args = arguments;
    timeoutId = setTimeout(() => {
      func.apply(context, args);
    }, delay);
  };
}

function sayHello(name) {
  console.log("Hello, " + name + "!");
}

const debouncedHello = debounce(sayHello, 1000);

// Simulate rapid calls
debouncedHello("Alice");
debouncedHello("Bob");
debouncedHello("Charlie"); // Only this will log after 1s

```

lets take an example to implement debounce for a search function

here we have html input tag <TextField/> on change when user types in text box it will call

debounce search

```

const performSearch = async (text) => {
  try {
    const response = await axios.get(

```

```

    `${url}/products/search?value=${text}`
  );
  setProductlist(response.data);
} catch (err) {
  setProductlist(null);
}
};

const debounceSearch = (event, debounceTimeout) => {
  if (debounceTimeout !== 0) {
    clearTimeout(debounceTimeout);
  }
  // need not to be async
  const timeout = setTimeout(async () => {
    performSearch(event);
  }, 500);
  setDebounceTimeout(timeout);
};

//html code
<TextField
  className="search-desktop"
  size="small"
  onChange={{e} => debounceSearch(e.target.value, debounceTimeout)}}
  InputProps={{
    endAdornment: (
      <InputAdornment position="end">
        <Search color="primary" />
      </InputAdornment>
    ),
  }}
  placeholder="Search for items/categories"
  name="search"
/>

```

In example above i am using arrow function in set timeout so i dont need apply or save this value

arrow function inherits **this context** from parent

now lets implement same search debounce using cleaner version and reusable

```
function debounce(func, delay) {  
  let timer; // timer is preserved in closure  
  
  return function (...args) {  
    const context = this;  
  
    clearTimeout(timer);  
  
    timer = setTimeout(() => {  
      func.apply(context, args); // run with original context and arguments  
    }, delay);  
  };  
}  
  
const performSearch = async (text) => {  
  try {  
    const response = await axios.get(  
      `${config.endpoint}/products/search?value=${text}`  
    );  
    setProductlist(response.data);  
  } catch (err) {  
    setProductlist(null);  
    enqueueSnackbar(  
      "Something went wrong. Check the backend console for more details",  
      { variant: "error" }  
    );  
  }  
};
```

```
// Create the debounced version once
const debouncedSearch = debounce(performSearch, 500);

//jsx
<TextField
  className="search-desktop"
  size="small"
  onChange={(e) => debouncedSearch(e.target.value)}
  InputProps={{
    endAdornment: (
      <InputAdornment position="end">
        <Search color="primary" />
      </InputAdornment>
    ),
  }}
  placeholder="Search for items/categories"
  name="search"
/>
```

9. Implement Throttling

Throttling means **run the function at most once every X milliseconds**, even if the event keeps firing.

◆ Use case:

- Scroll event: Avoid calling a function hundreds of times while scrolling.
- Button spam protection: Prevent a function from running repeatedly

```
function throttle(fn, limit) {
  let lastCall = 0;
```

```

return function (...args) {
  const now = Date.now();
  if (now - lastCall >= limit) {
    lastCall = now;
    fn.apply(this, args);
  }
};
}

// Usage:
window.addEventListener("scroll", throttle(() => {
  console.log("Scroll handler running");
}, 200));

```

10. Flatten a nested object

```

function flattenObject(obj, parentKey = '', res = {}) {
  for (let key in obj) {
    const propName = parentKey ? `${parentKey}.${key}` : key;
    if (typeof obj[key] === 'object' && !Array.isArray(obj[key])) {
      flattenObject(obj[key], propName, res);
    } else {
      res[propName] = obj[key];
    }
  }
  return res;
}

// flattenObject({ a: { b: { c: 1 } } }) => { "a.b.c": 1 }

```

11. Memoize a function

```
function memoize(fn) {
  const cache = {};

  return function (...args) {
    const key = JSON.stringify(args);
    if (cache[key]) return cache[key];
    const result = fn.apply(this, args);
    cache[key] = result;
    return result;
  };
}
```

12. What is **deepClone** ?

deepClone means to **create a completely independent copy of an object**, including all its nested objects or arrays — not just a shallow copy.

Unlike `Object.assign()` or the spread operator `{ ...obj }`, which only copy the top level (shallow), **deepClone** copies everything — deeply.

✅ The Code (Explained Simply)

```
function deepClone(obj) {
  // 1. Base case: if obj is null or not an object (i.e. it's a primitive), just return it directly
  if (obj === null || typeof obj !== 'object') return obj;

  // 2. If it's an array, recursively clone each item and return a new array
  if (Array.isArray(obj)) return obj.map(deepClone);
```

```
// 3. If it's a plain object, recursively copy each key-value pair
const copy = {};
for (let key in obj) {
  copy[key] = deepClone(obj[key]); // Recursively clone nested values
}

// 4. Return the fully copied object
return copy;
}
```

✓ Example

```
const original = {
  name: 'Alice',
  address: {
    city: 'Pune',
    geo: {
      lat: 18.5,
      lng: 73.8
    }
  },
  hobbies: ['reading', 'music']
};

const clone = deepClone(original);

clone.name = 'Bob';
clone.address.city = 'Mumbai';
clone.hobbies[0] = 'gaming';

console.log(original.name); // Alice ✓
console.log(original.address.city); // Pune ✓
```

```
console.log(original.hobbies[0]); // reading ✓
```

➡ All the original values stay the same — because the `clone` is **completely separate**.

or can use this

```
const clone = JSON.parse(JSON.stringify(original));
```

```
const clone = structuredClone(original);
```

using spread operator for objects will only create shallow copy not deep copy

```
const clone = { ...original };
```

13. Spread Operator for Arrays (`[...]`)

```
const original = [1, 2, 3];  
const copy = [...original];
```

What happens?

- This creates a **new array**, copying the **top-level elements**.
- So if elements are **primitives** (like numbers, strings), it's fine.

```
copy[0] = 999;  
console.log(original); // [1, 2, 3] ✓
```

But with Nested Arrays or Objects?

The **spread operator still does a shallow copy**.

```
const original = [[1, 2], [3, 4]];
const copy = [...original];

copy[0][0] = 999;

console.log(original[0][0]); // 999 ❌ — original is affected
```

Same issue with objects in arrays:

```
const original = [{ name: "Alice" }, { name: "Bob" }];
const copy = [...original];

copy[0].name = "Charlie";

console.log(original[0].name); // "Charlie" ❌
```